Golang Cheatsheet

Essential syntax and operations for Go programming

This cheatsheet provides a comprehensive reference to fundamental Go language constructs, syntax patterns, and core features. Find variable declarations, control flow structures, data types, and standard library operations for efficient Go programming.

Basic Syntax	Control Flow	Data Structures
Variables, types, and declarations	Conditionals, loops, and switch	Arrays, slices, maps, and structs
Functions	**Concurrency**	
Definition, parameters, and returns	Goroutines and channels	

Installation & Setup

Install Go (Ubuntu/Linux) Download and install Go from the official website.

```
# Download from https://golang.org/dl/
wget https://go.dev/dl/go1.21.5.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.21.5.linux-amd64.tar.gz

# Add to PATH in ~/.bashrc or ~/.zshrc
export PATH=$PATH:/usr/local/go/bin

# Verify installation
go version
```

Package Manager: Using Homebrew (macOS)

```
# Install Go
brew install go

# Update Go
brew upgrade go

# Uninstall Go
brew uninstall go
```

Windows Installation Install Go on Windows systems.

```
# Download .msi installer from https://golang.org/dl/
# Run installer and follow prompts

# Verify installation
go version

# Update PATH if needed
set PATH=%PATH%;C:\Go\bin
```

Basic Syntax & Types

Core language constructs and data types for fundamental programming.

Package & Imports

```
package main

import "fmt"
import (
    "os"
    "strings"
)

// Single import
import "io"
```

Variables & Constants

```
var name string = "John"
var age int = 25
var isActive bool

// Short declaration
name := "Jane"
age := 30

// Constants
const Pi = 3.14159
const Message = "Hello, Go"
```

Basic Data Types

```
// Primitive types
int, int8, int16, int32, int64
uint, uint8, uint16, uint32, uint64
float32, float64
bool
string

// Type conversion
var i int = 42
var f float64 = 3.14
var b byte = byte(i)
var s string = string(65) // "A"

// Type aliases
type MyInt int
type Celsius = float64
var temp Celsius = 25.5
```

Control Flow

Conditionals: `if` / `else` / `switch`

```
// Simple if statement
if x > 10 {
    fmt.Println("x is greater than 10")
} else if x > 5 {
    fmt.Println("x is greater than 5")
} else {
    fmt.Println("x is 5 or less")
}

// Switch statement
switch day {
    case "Monday", "Tuesday":
        fmt.Println("Start of work week")
    case "Friday":
        fmt.Println("End of work week")
    default:
        fmt.Println("Regular day")
}
```

Loops: `for` / `range`

```
// Classic for loop
for i := 0; i < 10; i++ {
    fmt.Printf("%d ", i)
}

// While-style loop
for condition {
    // do something
}

// Infinite loop
for {
    // Break when condition
}
```

Functions

Function Declaration: `func`

```
// Basic function
func greet(name string) {
    fmt.Printf("Hello, %s!\n", name)
}

// Multiple return values
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("division by zero")
    }
    return a / b, nil
}

// Named return values
func rectangle(width, height int) (area, perimeter int) {
    area = width * height
    perimeter = 2 * (width + height)
    return // returns area and perimeter by name
}

// Variadic function
func sum(numbers ...int) int {
    total := 0
    for _, n := range numbers {
        total += n
    }
    return total
}

// Usage
greet("Alice")
result, err := divide(10, 2)
if err != nil {
    log.Fatal(err)
}
area, perimeter = rectangle(5, 3)
total = sum(1, 2, 3, 4, 5)
```

Function Literals & Closures

```
// Anonymous function
func(x int) {
    fmt.Printf("Value: %d\n", x)
}(42)

// Closure
func makeCounter() func() int {
    count := 0
    return func() {
        count++
        return count
    }
}

// Usage
counter := makeCounter()
fmt.Printf("%d\n", counter())
fmt.Printf("%d\n", counter())
```

Defer Statement

```
// Defer executes after surrounding function returns
func processFile(filename string) {
    file, err := os.Open(filename)
    if err != nil {
        return
    }
    defer file.Close() // Will close file when function returns

    // Process the contents
    // File.Close() will be called automatically
}
```

Data Structures

Arrays, slices, maps, and structs for organizing data.

Arrays & Slices

```
// Arrays
var arr [5]int = [5]int{1, 2, 3, 4, 5}
arr[0] = 10
fmt.Printf("%d\n", arr[0])

// Slices (dynamic)
var slice []int
slice = append(slice, 1, 2, 3)

// Slice operations
slice = slice[1:3]  // [2, 3]
slice = slice[:2]   // [1, 2]
slice = slice[1:]   // [2]

// Make slice with capacity
slice = make([]int, 5, 10)
fmt.Printf("len: %d, cap: %d\n", len(slice), cap(slice))

// Slice operations
slice = append(slice, 6, 7, 8)
slice = append(slice, otherSlice...)
copy(destSlice, srcSlice)
```

Maps

```
// Maps
var m map[string]int
m = make(map[string]int)
m["key1"] = 10
m["key2"] = 20

// Map operations
value = m["key1"]
value, ok = m["key1"]
if ok {
    fmt.Printf("Found: %d\n", value)
}

delete(m, "key1")

// Iterate over map
for key, value := range m {
    fmt.Printf("%s: %d\n", key, value)
}
```

Structs

```
// Struct definition
type Person struct {
    Name string
    Age  int
    Email string
}

// Struct literal
p := Person{
    Name: "John",
    Age: 30,
    Email: "john@example.com",
}

// Anonymous struct
point := struct {
    x, y int
}{10, 20}

// Struct methods
func (p Person) IsAdult() bool {
    return p.Age >= 18
}

func (p *Person) SetAge(age int) {
    p.Age = age
}

// Usage
if p.IsAdult() {
    fmt.Printf("%s is adult\n", p.Name)
}
p.SetAge(25)
```

Methods & Interfaces

Methods

```
// Value receiver
func (p Person) String() string {
    return fmt.Sprintf("%s (%d)", p.Name, p.Age)
}

// Pointer receiver (can modify)
func (p *Person) Birthday() {
    p.Age++
}

// Usage
p := Person{Name: "Alice", Age: 25}
fmt.Printf("%s\n", p.String())
p.Birthday()
fmt.Printf("%s\n", p.String())
```

Interfaces

```
// Interface definition
type Stringer interface {
    String() string
}

type Reader interface {
    Read(p []byte) (n int, err error)
}

// Implement interface (no explicit declaration)
// Any type with these methods implements the interface

// Empty interface (accepts any type)
var anything interface{}
anything = 42
anything = "hello"

// Type assertion
str, ok := anything.(string)
if ok {
    fmt.Printf("%s\n", str)
}

// Type switch
switch v := anything.(type) {
    case int:
        fmt.Printf("%d\n", v)
    case string:
        fmt.Printf("%s\n", v)
    default:
        fmt.Printf("unknown type\n")
}
```

Embedding

```
// Struct embedding
type Employee struct {
    Person
    Salary float64
    Company string
}

// Usage
e := Employee{
    Person: Person{Name: "Alice", Age: 30},
    Salary: 75000.0,
    Company: "TechCorp",
}
fmt.Printf("%s\n", e.String())

// Access embedded fields directly
fmt.Printf("%s\n", e.Name)
```

Empty Interface & Type Assertions

```
// Work with values of unknown type
func processValue(v interface{}) {
    switch v.(type) {
    case int:
        fmt.Printf("Integer: %d\n", v.(int))
    case float64:
        fmt.Printf("Float: %.2f\n", v.(float64))
    case string:
        fmt.Printf("String: %s\n", v.(string))
    case bool:
        fmt.Printf("Boolean: %t\n", v.(bool))
    default:
        fmt.Printf("Unknown type\n")
    }
}

// Type assertion with error checking
str, ok := v.(string)
if !ok {
    return fmt.Errorf("expected string, got %T", v)
}
fmt.Printf("%s\n", str)

// Multiple type assertions
if1, ok1 := v.(interface{ Method() int })
if2, ok2 := v.(interface{ Method() int })
if ok1 && ok2 {
    fmt.Printf("%d %d\n", if1.Method(), if2.Method())
}
```

Error Handling

Handle errors gracefully using Go's error interface.

Basic Error Handling

```
// Check error immediately
file, err := os.Open("filename.txt")
if err != nil {
    log.Fatal(err)
}
defer file.Close()

// Ignore specific errors
_, err = fmt.Fprintf(os.Stdout, "Hello\n")
if err != nil {
    // Handle error
}

// Multiple return values
value, err := strconv.Atoi("123")
if err != nil {
    fmt.Printf("Error: %s\n", err.Error())
    return
}
fmt.Printf("Value: %d\n", value)
```

Custom Errors

```
// Simple error
var ErrNotFound = errors.New("not found")

// Formatted error
var ErrInvalidInput = fmt.Errorf("invalid input")

// Check specific error
if err == ErrNotFound {
    fmt.Printf("Not found\n")
}

// Wrap errors (Go 1.13+)
var ErrDatabase = fmt.Errorf("database error: %w", ErrNotFound)

// Check wrapped error
if errors.Is(err, ErrNotFound) {
    fmt.Printf("Not found\n")
}
```

Concurrency

Goroutines and channels for concurrent programming.

Goroutines

```
// Start goroutine
go processData()

// Anonymous goroutine
go func() {
    fmt.Printf("Hello from goroutine\n")
}()

// Wait for goroutines to finish
var wg sync.WaitGroup
wg.Add(1)
go func() {
    defer wg.Done()
    // Work
}()
wg.Wait()
```

Channels

```
// Create channel
ch := make(chan int)

// Send to channel
ch <- 42

// Receive from channel
value := <-ch

// Buffered channel
ch = make(chan int, 10)

// Close channel
close(ch)

// Check if channel closed
value, ok := <-ch
if !ok {
    fmt.Printf("Channel closed\n")
}

// Range over channel
for value := range ch {
    fmt.Printf("%d\n", value)
}

// Select statement
select {
    case ch1 <- value1:
        fmt.Printf("Sent to ch1\n")
    case value2 := <-ch2:
        fmt.Printf("Received %d\n", value2)
    default:
        fmt.Printf("No channel ready\n")
}
```

Advanced Topics

Pointers, JSON processing, HTTP servers, and more advanced features.

Pointers

```
// Pointer declaration
var p *int
i := 42
p = &i
fmt.Printf("%d\n", *p)

// Pointer to struct
var person *Person
person = &Person{Name: "Alice", Age: 30}
person.Age++

// Pointer receiver modifies original
func (p *Person) Birthday() {
    p.Age++
}

// Nil pointer checking
if p != nil {
    fmt.Printf("%s\n", p.Name)
}
```

JSON Processing

```
// Struct with JSON tags
type Person struct {
    Name  string `json:"name"`
    Age   int    `json:"age"`
    Email string `json:"email,omitempty"`
}

// Marshal (to JSON)
person := Person{Name: "Alice", Age: 30}
jsonBytes, err := json.Marshal(person)
if err != nil {
    log.Fatal(err)
}
fmt.Printf("%s\n", jsonBytes)

// Unmarshal (from JSON)
var p Person
err = json.Unmarshal(jsonBytes, &p)
```

HTTP Server

```
// Simple HTTP server
func handler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, %s\n", r.URL.Path[1:])
}

func main() {
    http.HandleFunc("/", handler)
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Benchmarking

```
// Benchmark function
func BenchmarkSum(b *testing.B) {
    for i := 0; i < b.N; i++ {
        sum(1, 2, 3, 4, 5)
    }
}

// Run benchmarks
// go test -bench=.
// go test -bench=BenchmarkSum -benchmem
```

Example Tests

```
// Example function (appears in documentation)
func ExampleSum() {
    fmt.Printf("%d\n", sum(1, 2, 3))
    // Output: 6
}

// Run examples
// go test -run=Example
```

Testing Fundamentals

```
// Basic test function
func TestSum(t *testing.T) {
    result := sum(1, 2, 3)
    if result != 6 {
        t.Errorf("sum(1, 2, 3) = %d; expected 6", result)
    }
}

// Table-driven tests
func TestSum(t *testing.T) {
    tests := []struct {
        name     string
        input    []int
        expected int
    }{
        {"positive", []int{1, 2, 3}, 6},
        {"zero", []int{0, 0, 0}, 0},
        {"negative", []int{-1, -2, -3}, -6},
        {"single", []int{5}, 5},
        {"empty", []int{}, 0},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result := sum(tt.input...)
            if result != tt.expected {
                t.Errorf("sum(%v) = %d; expected %d",
                    tt.input, result, tt.expected)
            }
        })
    }
}

// Run specific test
// go test -run=TestSum

// Run with verbose output
// go test -v

// Run benchmarks
// go test -bench=.

// Run tests with coverage
// go test -cover
```

Go Modules & Packages

Manage dependencies and create reusable packages.

Module Management

```
// Initialize new module
// go mod init github.com/username/project

// Add dependency
// go get github.com/gorilla/mux

// Update specific dependency
// go get github.com/gorilla/mux@v1.8.0

// Remove unused dependencies
// go mod tidy

// Download dependencies
// go mod download

// Verify dependencies
// go mod verify

// Show dependency graph
// go mod graph
```

go.mod File

```
// Contents of go.mod
module github.com/username/project

go 1.21

require (
    github.com/gorilla/mux v1.8.0
    golang.org/x/crypto v0.17.0 // indirect
)

// Local replacement (for development)
replace (
    github.com/mycompany/mylib => ../mylib
)

// Exclude specific version
exclude (
    golang.org/x/crypto v0.16.0
)
```

Creating Packages

```
// Package comment
// This package provides utilities for...
package utils

// Exported function (starts with capital letter)
func ExportedFunction() {
    fmt.Printf("Public function\n")
}

// Private function (starts with lowercase)
func helperFunction() {
    fmt.Printf("Helper\n")
}

// Exported type
type MyStruct struct {
    ExportedField int
    privateField  int
}

// Exported constant
const MaxSize = 100

// Exported variable
var DefaultConfig = "default"
```

Common Go Commands

```
// Run program
go run main.go

// Build executable
go build
go build -o myapp .

// Install to $GOPATH/bin
go install

// Format code
go fmt ./...

// Vet code for issues
go vet ./...

// Check build cache
go clean -cache
```


Attribution: This cheatsheet is maintained by contributors and is intended for quick reference purposes. Contributions are welcome through pull requests. Please verify critical syntax and behaviors against official documentation.